

## Blocco 8 - Subquery e logica insiemistica

Esprimere condizioni su insiemi di righe

Relational Databases & SQL

30 min teoria + 90 min pratica

# **1. Rappresentazione iniziale**

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

# Perché questo blocco

## Problema didattico

Alcune domande non chiedono solo di collegare tabelle, ma di verificare esistenza, assenza o confronto con un insieme: clienti che hanno comprato almeno una volta, prodotti mai venduti, ordini sopra la media.

## Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

## Situazione concreta

Per trovare prodotti mai venduti potremmo esportare due liste e confrontarle manualmente. Funziona su pochi dati, ma non è robusto.

| lista prodotti | lista prodotti venduti |
|----------------|------------------------|
| P10, P11, P12  | P10, P12               |

# Limiti della rappresentazione iniziale

- ▶ procedura manuale;
- ▶ errore facile con duplicati;
- ▶ NULL può cambiare il risultato;
- ▶ non scala a dati reali.

## 2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

# Da confronto manuale a logica insiemistica

## Principio

Subquery e operatori insiemistici permettono di esprimere esistenza, assenza e confronto con insiemi prodotti dal database.

# Le parole chiave della domanda

| Frase di dominio            | Forma logica            | Costrutto tipico |
|-----------------------------|-------------------------|------------------|
| ha almeno un ordine         | esiste almeno una riga  | EXISTS           |
| non ha mai ordinato         | non esiste alcuna riga  | NOT EXISTS       |
| è sopra la media            | confronto con un valore | subquery scalare |
| è presente in A ma non in B | differenza tra insiemi  | EXCEPT           |

## Idea

Prima si riconosce il quantificatore, poi si sceglie la sintassi.



## Subquery: domanda annidata

- ▶ la query esterna decide la granularità del risultato finale;
- ▶ la query interna produce un valore o un insieme di confronto;
- ▶ una subquery correlata usa una colonna della riga esterna corrente;
- ▶ una subquery non correlata può essere eseguita e capita da sola.

# Esempio di riorganizzazione

```
SELECT p.product_id, p.sku
FROM products p
WHERE NOT EXISTS (
  SELECT 1 FROM order_items oi
  WHERE oi.product_id = p.product_id
);
```

- ❶ identificare il soggetto esterno;
- ❷ riconoscere il quantificatore: almeno uno, nessuno, tutti;
- ❸ scegliere EXISTS o NOT EXISTS;
- ❹ testare la subquery;
- ❺ controllare il comportamento con valori mancanti.

# Come leggere EXISTS

## Lettura riga per riga

Per ogni riga della query esterna, il DBMS prova la subquery interna. Se la subquery produce almeno una riga, EXISTS è vero.

- ▶ non importa quali colonne restituisce la subquery;
- ▶ per convenzione si usa `SELECT 1`;
- ▶ la condizione di correlazione collega interno ed esterno;
- ▶ l'ordinamento nella subquery non serve per EXISTS.

# NOT EXISTS: assenza come condizione

## Esempio mentale

“Clienti senza ordini” significa: per quel cliente non deve esistere alcuna riga in `orders`.

## Punto delicato

L'assenza non è un valore `NULL`: è la mancanza di righe collegate.

## Analogia o caso esplicativo

Per sapere chi deve essere sollecitato, si confronta la lista di tutti i clienti con la lista di chi ha già ordinato. La logica insiemistica rende esplicite inclusioni, esclusioni e sovrapposizioni.

### 3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

# Formalizzazione: subquery e insiemi

- ▶ subquery scalare;
- ▶ subquery tabellare;
- ▶ subquery correlata;
- ▶ EXISTS e NOT EXISTS;
- ▶ UNION, INTERSECT, EXCEPT.



- ▶ **Subquery scalare:** restituisce un valore.
- ▶ **Subquery tabellare:** restituisce un insieme di righe.
- ▶ **Subquery correlata:** usa valori della query esterna.
- ▶ **EXISTS:** vero se la subquery produce almeno una riga.
- ▶ **EXCEPT:** differenza tra due insiemi compatibili.

# Subquery scalare: una cella come risultato

- ▶ restituisce una sola riga e una sola colonna;
- ▶ è utile per confronti con soglie calcolate;
- ▶ esempio: prezzo maggiore della media;
- ▶ se restituisce più righe, PostgreSQL segnala errore.

# Esempio: prezzo sopra media

```
SELECT product_id, sku, product_name, unit_price
FROM products
WHERE unit_price > (
    SELECT avg(unit_price)
    FROM products
    WHERE active = true
)
ORDER BY unit_price DESC;
```

La subquery interna produce una soglia; la query esterna decide quali prodotti mostrare.

# IN, NOT IN e il problema dei NULL

- ▶ IN confronta un valore con un insieme di valori;
- ▶ NOT IN sembra naturale per le assenze;
- ▶ se l'insieme contiene NULL, il risultato può diventare non intuitivo;
- ▶ per le assenze collegate a tabelle reali, preferire spesso NOT EXISTS.

# Come riconoscere una buona soluzione

- ▶ il soggetto della query esterna è chiaro;
- ▶ il quantificatore business è espresso dalla clausola corretta;
- ▶ la subquery è testabile anche da sola quando possibile;
- ▶ NULL e assenze sono gestiti intenzionalmente.

## 4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

## Esempio: situazione iniziale

### Domanda o frase di dominio

Domanda: “Quali clienti hanno almeno un ordine con ricavo superiore alla media degli ordini validi?”

# Esempio: ragionamento

- ▶ soggetto esterno: clienti;
- ▶ condizione: esiste un ordine del cliente;
- ▶ confronto: ricavo ordine maggiore della media globale;
- ▶ popolazione media: ordini non cancellati e non rimborsati.



# Esempio: prima isolare la media

```
SELECT avg(gross_revenue) AS average_order_revenue  
FROM order_revenue  
WHERE status NOT IN ('cancelled', 'refunded');
```

Una subquery non correlata va capita prima da sola: restituisce una soglia globale.

# Esempio completo: EXISTS e media

```
SELECT c.customer_id, c.full_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM order_revenue r
    WHERE r.customer_id = c.customer_id
        AND r.status NOT IN ('cancelled', 'refunded')
        AND r.gross_revenue > (
            SELECT avg(gross_revenue)
            FROM order_revenue
            WHERE status NOT IN ('cancelled', 'refunded')
        )
)
ORDER BY c.full_name;
```

# Esempio: ordini senza pagamento catturato

```
SELECT o.order_id, o.order_date, o.status
FROM orders AS o
WHERE NOT EXISTS (
    SELECT 1
    FROM payments AS p
    WHERE p.order_id = o.order_id
        AND p.status = 'captured'
)
ORDER BY o.order_id;
```

La domanda non è “payment nullo”, ma “non esiste un pagamento catturato per quell’ordine”.

# Esempio: differenza con EXCEPT

```
SELECT customer_id
FROM customers
EXCEPT
SELECT DISTINCT customer_id
FROM support_tickets
ORDER BY customer_id;
```

EXCEPT è leggibile quando vogliamo mostrare esplicitamente “A meno B”.

# Quando scegliere cosa

- ▶ JOIN: quando servono colonne di più tabelle nello stesso risultato.
- ▶ EXISTS: quando serve verificare l'esistenza di almeno una riga.
- ▶ NOT EXISTS: quando la domanda parla di assenza.
- ▶ Subquery scalare: quando serve confrontare con una soglia calcolata.
- ▶ EXCEPT: quando la richiesta è una differenza tra due insiemi compatibili.

# Errori frequenti da prevenire

- ▶ usare NOT IN senza pensare a NULL;
- ▶ scrivere subquery che restituiscono più righe dove serve un solo valore;
- ▶ non distinguere query correlata e non correlata;
- ▶ usare subquery quando un join sarebbe più leggibile;
- ▶ dimenticare filtri di stato nella subquery.

## 5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

# Esercitazione: problema informale

## Contesto

Marketing e operations chiedono segmenti particolari: clienti con ordini sopra media, prodotti mai venduti, clienti senza ticket e ordini senza pagamento.



## Esercitazione: specifica corretta

- ▶ input: tabelle e vista `order_revenue`;
- ▶ output: query con subquery o operatori insiemistici coerenti con la frase;
- ▶ vincolo: usare `NOT EXISTS` per assenze quando ci sono possibili `NULL`;
- ▶ vincolo: motivare la scelta tra join, subquery e operatore insiemistico.

# Esercitazione: definizione della soluzione

- ① partire dal soggetto della domanda;
- ② scrivere la subquery interna separatamente quando non è correlata;
- ③ trasformare “non hanno” in `NOT EXISTS`;
- ④ usare `EXCEPT` per differenze esplicite tra insiemi.

# Implementazione completa: prodotti mai venduti (1/2)

```
SET search_path TO training;

SELECT c.customer_id, c.full_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM order_revenue r
    WHERE r.customer_id = c.customer_id
        AND r.status NOT IN ('cancelled', 'refunded')
        AND r.gross_revenue > (
            SELECT avg(gross_revenue)
            FROM order_revenue
            WHERE status NOT IN ('cancelled', 'refunded')
        )
)
ORDER BY c.full_name;
```

## Implementazione completa: prodotti mai venduti (2/2)

```
SELECT p.product_id, p.sku, p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
)
ORDER BY p.sku;

SELECT customer_id
FROM customers
EXCEPT
SELECT DISTINCT customer_id
FROM support_tickets
ORDER BY customer_id;
```

# Implementazione completa: assenza di pagamenti

```
SELECT o.order_id, o.customer_id, o.order_date, o.status
FROM orders AS o
WHERE o.status NOT IN ('cancelled', 'refunded')
      AND NOT EXISTS (
        SELECT 1
        FROM payments AS p
        WHERE p.order_id = o.order_id
              AND p.status = 'captured'
      )
ORDER BY o.order_date, o.order_id;
```

# Implementazione completa: sopra la media per segmento

```
SELECT c.customer_id, c.full_name, c.segment
FROM customers AS c
WHERE EXISTS (
  SELECT 1
  FROM order_revenue AS r
  WHERE r.customer_id = c.customer_id
    AND r.gross_revenue > (
      SELECT avg(r2.gross_revenue)
      FROM order_revenue AS r2
      JOIN customers AS c2
        ON c2.customer_id = r2.customer_id
      WHERE c2.segment = c.segment
    )
);
```

# Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min riconoscere quantificatori nelle tracce;
- ▶ 25 min subquery scalari e confronti con media;
- ▶ 25 min EXISTS e NOT EXISTS;
- ▶ 15 min operatori insiemistici;
- ▶ 10 min confronto su alternative equivalenti.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.



- ▶ subquery e insiemi servono quando la domanda parla di esistenza, assenza o confronto;
- ▶ EXISTS legge la domanda come “c'è almeno una riga?”;
- ▶ le assenze vanno trattate con cura per non confondere NULL e mancata corrispondenza.